

PHYSICAL TOUCH OF VIRTUAL OBJECTS

ROKAS ANDRIJAUSKAS 4129184

Masters Project in Engineering (Robot Systems)
Supervisor: Xiaofeng Xiong

June 2025



The Maersk Mc-Kinney Moeller Institute
University of Southern Denmark

Character count: 68161

Abstract

Many companies are struggling to combine Robotics and Machine Learning . .

Contents

Contents	ii
1 Introduction	1
1.1 Background	1
1.2 Problem statement	1
1.3 Objective	2
1.4 Research question	2
2 Exoskeleton	3
2.1 Background	3
2.2 Analysis and Design 1	11
2.3 Implementation 1	12
2.4 Analysis and Design 2	19
2.5 Conclusion	20
3 Algorithms	22
3.1 Background	22
3.2 Analysis and Design	34
3.3 Implementation	34
3.4 Conclusion	34
4 Discussion	35
5 Conclusion	36
A Appendix	37
Bibliography	38

1 Introduction

1.1 Background

Recent advancements in computer hardware and software have enabled nearly photorealistic and real-time simulation of virtual worlds. Technologies such as augmented reality (AR) and virtual reality (VR) enhance the immersion of these simulated world by overlaying information over the user's field of view. Since the technology is still in its early stages, there is still a lot of unsolved problems and progress to be made. The immersion into virtual worlds is usually achieved through the use of a headset. Early headsets were really bulky, had low resolution and short battery life. Naturally most of the research was focused on tackling these problems. In the present day we have consumer grade devices like meta quest 3 and apple vision pro that offer excelent battery life, sharp visuals and immersive audio. In the future we could expect this technology to shrink to the size of regular pair of glasses. Resarchers at Meta are already stepping in that direction with ther recently unveiled Orion glasses.

The hardware improvements also made the creation of Artificial Intelligence (AI) possible. It was theorized as early as 1950s by Alan Turing if machines are capable of thinking [39]. In 1959 Arthur Samuel coined the term 'machine learning', but Ii was only in 1990s that hardware and algorithms were sufficiently advanced that machine learning began to gain traction. Due to well-defined environments and rules, games were the first domain to notice the impact of AI. In 1997, Deep Blue [4] managed to beat the world chess champion Garry Kasparov. As computers following Moore's law [29] grew in power, more breakthroughs began to emerge. Currently, in 2025, the AI is capable creating images, videos, music, text, research podcasts and even interactive worlds.

The artificial intelligence also gave rise to the robotics industry. AI is transforming robots from rigid, pre-programmed machines into intelligent systems capable of performing increasingly complex tasks. Robots are often trained in simulated environments as this allows for faster and safer training without needing to worry about demaging robot or the environment.

1.2 Problem statement

Despite the fact that VR and AR technologies are rapidly improving, there is still a major problem, these technologies usually immerse only two senses: sight

and hearing, but we, as humans, have five: sight, hearing, taste, smell, and touch. This means that there is still a lot of room for improvement as inclusion of additional senses would provide a more immersive VR/AR experience.

1.3 Objective

The thesis aims to address the limitation of VR technology by incorporating the sense of touch into the computer-simulated virtual worlds. There are two objectives, first, following engineering design process, will be focused on developing a portable exoskeleton capable of giving its user an immersive sense of touch in VR. Second, will be dedicated to exploring if the developed exoskeleton could aid in the process of training robots in simulated world.

1.4 Research question

Can a portable exoskeleton that gives its users the ability to touch virtual objects be created in a way that it is a viable solution to aid in the training of the robots in the simulated environments?

2 Exoskeleton

This chapter delves into the research, development and testing of the portable exoskeleton.

2.1 Background

In this section a comparison of physics world simulators and state of the art exoskeletons that attempt to incorporate touch into virtual environments will be made.

Virtual environment

The hard requirements for the choice of the simulator for this thesis are simple, it must have:

1. VR support. Users should be able to interact with the environment.
2. Support of the soft-body physics.

A great to have features for the simulator are the following:

1. Free and open-source, so the research in this thesis could be easily accessible and replicated.
2. Fast, so the development would go smoothly on an affordable computer.
3. Popular, it is easier to fix problems if we can find people in forums who already had them.
4. Easy to learn, it would be great to have more time on research and implementation than learning the engine.
5. Used in robotics applications.

Numerous physics engines have been developed in which virtual environments can be created. Most notable are [7]: Mujoco, Gazebo, CoppeliaSim, Webots, PyBullet and SOFA.

Multi-Joint dynamics with Contact (MuJoCo) [38] is a popular physics engine that required a commercial license up until recently. In 2021 Google Deepmind acquired the simulator and made it freely available for everyone [9]. In 2022, they open-sourced and published it on Github. The MuJoCo is widely used

in research due to its contact stability [41]. State of the art research leverages MuJoCo for training robotic manipulator policies in simulation. It is worth noting that MuJoCo lacks built-in functionality for inverse kinematics or path planning.

Gazebo is another popular robotics simulator that is widely used in research. One of the things that makes it so appealing is support of Robotics Operating System [31] (ROS). ROS houses a expansive set of libraries and tools for building robotics applications. ROS-based software is platform and language agnostic making incorporation of user developed libraries very easy. The working principle that allows for this seamless integration is Publish/Subscribe model. ROS processes called nodes, communicate between each other by publishing messages to topics and receiving messages from subscribed topics allowing interprocess communication. A study has shown that if the robot model in Gazebo was created properly, the code that was used in simulation can be directly reused in real robot without any modifications [37]. Unfortunately the engine does not support soft-body physics natively. There are however attempts to bridge this gap between rigid-body and soft-body physics. One study developed a ROS-Gazebo Toolbox that allows user to bring Articulated Soft Robots in the simulation with satisfactory Sim2Real performance [26]

CoppeliaSim, formerly known as V-REP [34], has an easy learning curve with it's GUI driven design and user- centric feature including forward and inverse kinematics, sensor simulation, motion and path planning. Built around a distributed control architecture, the engine supports scripts written in Lua or Python and plugins written in C/C++ that act as synchronous controllers. This is useful in multi-robot setups. It also has ROS support, but unlike Gazebo, it is secondary to the engine's design. CoppeliaSim supports 5 different physics engines (MuJoCo, Bullet Physics, ODE, Newton and Vortex Dynamics), but the support for soft body and deformable objects is lacking.

Webots [27], started in 1996, is an open-source simulator witten in C++. It has a large collection of sensors, actuator and robot models. The simulator is beginner friendly with polished GUI that makes it easy to build models from scratch. One study comparing robot simulators has shown that Webots was least resource intensive, it the lowest CPU usage compared to CoppeliaSim and Gazebo [1]. These reasons make the simulator appealing for fast prototyping and multi-robot simulation purposes. Despite these great features the simulator lacks soft-body dynamics support.

Bullet is a professional library written in C++. It is videly used in games, visual effects and robotics applications. It can simulate Soft-Body as well as Rigid-Body physics with discrete and continuous collision detection. The PyBullet is a library with Python bindings that interact with the Bullet engine.

Simulator	Force Sensor	Inverse Kinematics	Soft-Body	VR	ROS
MuJoCo	✓	✗	✓	✓	✗
Gazebo	✓	✓	✗	✓	✓
CoppeliaSim	✓	✓	✗	✓	✓
Webots	✓	✗	✗	✓	✓
PyBullet	✓	✓	✓	✓	✗

Table 2.1: Feature Comparison of Popular Physics Simulators for Robotics Applications

This approach merges best of both worlds, the C++ speed and Python’s ease of use. PyBullet also has reinforcement learning, robotics simulation and VR support.

From the examined physics engines only MuJoCo and PyBullet support deformable objects and virtual reality. PyBullet was chosen as the physics engine for this thesis as it has a well documented VR section and broader VR support. Teleoperation with MuJoCo is only available through the use of MuJoCo HAPTIX [20] whereas PyBullet support a variety of VR controllers.

Exoskeletons

The human skin is the largest organ of the body, it plays a crucial role in the perception of touch and temperature. The density of touch receptors varies significantly across different skin regions. The palms of the hand, especially fingertips are one of the most touch sensitive parts of the body. These areas have a heightened sensitivity, because hands serve as the primary interface for interacting with the environment, feeling the objects temperature, texture and weight. Given the complexity and cost associated with the development of a full body exoskeleton, the focus of this thesis will exclusively be on create an exoskeleton for the hand only. There have already been multiple attempts to create an exoskeleton that enable user to feel the virtual objects. Some of which have even become commercially available.

Types of Feedback

There are multiple forms of haptic feedback a simple touch can provide. This section explore the different types of feedback and their significance in virtual interactions.

Transient contact and normal force play a fundamental role in perceiving the stiffness of an object. One study has shown that transient vibration combined with visuo-haptic illusion could successfully simulate this form of contact [6]. The results indicate that this type of contact can be modeled without requiring complex mechatronic or control systems.

Lateral stretch of the skin provide information about the weight of the object, while the stick-slip condition offers insight into when an object is about to be dropped. This information allows people to instinctively react by either gripping the object tighter to prevent slippage or try to catch it mid-air if it has already slipped. Various approaches and design methodologies have been investigated to replicate these tactile sensations in virtual environments [28][5].

Beyond force, pressure and friction feedback, the human skin is also adept at recognizing the texture of the objects. One study has shown that it is feasible to simulate texture perception using vibratory-only feedback by leveraging the spatial information of skin interactions with textured surfaces [40]. This suggests that a combination of finely tuned vibrations and high-resolution sensor data could enable realistic textural perception in virtual environments.

Temperature feedback is another important component of realistic haptic interactions. Temperature can provide important information about the type of material we are in contact with. Different heat flows and temperature gradients are experienced when touching objects due to the different thermal conductivity coefficients between materials. For example, when touching a metal or a wooden surface, even if both objects are at the same temperature, the metal will feel significantly colder than the wood. This is because the metal has much greater thermal conductivity than the wood. A wearable device that achieves temperature feedback has been proposed for the VR setting [13]. It contains a film with high thermal conductivity controller by Peltier element, which is coupled with a water-cooling heatsink in order to achieve the dynamic temperature simulation.

Although various studies have proposed solutions that simulate the individual aspects of the sense of touch, integrating multiple feedback modalities into a single, compact and lightweight exoskeleton remains a formidable engineering challenge. Combining force, texture, temperature, and friction feedback into a light, ergonomic and mass produced glove could be the goal for the future of haptic systems, however this thesis will exclusively focus on incorporating the sense the normal force exerted by the objects, as this is arguably the most important aspect of the touch as it allows users to feel the presence of the object.

Tracking

Hand tracking can be divided into two main categories: global position tracking and local position tracking. Global position tracking determines the whereabouts of the hand's location in the world space and local position tracking provides detailed information on the configuration and movement of the hand. Several technologies exist that can be attached to the exoskeleton to provide global position tracking.

Vive Tracker, the first commercially available VR tracker developed by HTC enable object and body tracking in virtual reality environments. They are compact, their dimensions are $70.9 \times 79.0 \times 44.1$ mm, lightweight, weighting only 75g and fully wireless, with battery life of up to 7.5 hours. These trackers can be attached to various body parts, such as hands, feet, or the waists, enabling accurate tracking of position and orientation in three-dimensional space. Their adaptability makes them a convenient off the shelf solution at a reasonable price point. The current market price is around 130€ per piece.

OptiTrack is a high-precision camera-based tracking system. They track the position of special reflective markers with a great accuracy. These markers are designed to reflect the infrared light emitted by OptiTrack camera system. By affixing these special reflective markers to an object of interest, the object can be successfully tracked. This method does not require a battery source on the exoskeleton, the marker are light, weighting only a couple of grams and they have no mechanical parts and the system offers a sub-millimeter precision. This method could provide an unparalleled user experience, but it does cost thousands and require significant setup in the room, making it a suitable option for research and industry applications rather than commercial use case.

To minimize costs and leverage existing VR hardware, some exoskeleton designs opt to incorporate standard VR controllers for global position tracking. These type of exoskeletons typically have a mount specifically designed for some type of VR controller, such as those used in HTC Vive or Oculus systems. This comes at almost no additional cost, but it also has significant the drawbacks. Firstly, the mount, buttons and other components inside the controller add unnecessary weight, secondly the form factor is designed to fit snugly in the palm of the hand, rather than be placed on the back of the hand, adding unnecessary bulk. These factors can potentially impact user's comfort and dexterity.

The local position tracking methods vary in complexity and accuracy. Valve Index Knuckles controllers utilize a proximity sensors to detect whether a user's finger is touching the controller or not. This allows for basic finger tracking, but it is limited in only 5 Degrees of Freedom (DOF) where each finger is in a binary state, either extended or flexed. This limits the realism of virtual hand interactions.

Data gloves provide a more refined finger tracking solution, offering continuous finger state tracking. Some data gloves [10] achieve that using flex capacitive stretch sensors. These sensors work due to their physical properties that resistance changes based on mechanical deformation. While they offer an accurate tracking, due to the nature how these sensors work, they are susceptible to

wear and thus have a limited life cycle. Modern flex capacitive stretch sensors achieve life cycle of over 1 million cycles, making their wear less of an issue. Some data gloves [23] achieve local position tracking using multiple inertial measurement units. This approach has an advantage of improved life cycle, as the sensors are purely electronic, but these type of gloves do experience “data drifting” which requires frequent recalibration. Moreover the reliability of IMU sensor is compromised in environments with strong or nearby magnetic fields making them unreliable to use with motor based exoskeletons.

Vision-based tracking solutions, such as Leap Motion, offer the highest number of degrees of freedom. These vision systems provide highly detailed motion capture without relying on extra mechanical or electrical sensors on the hand. However, these systems do only provide the tracking if the hands are in camera’s field of view and may loose accuracy in cases where the view of the hands is obstructed. This could be a problem with bulky exoskeletons, where the configuration of the hand is difficult to determine from the vision alone. This proves a need for further validation of the reliability of vision-based systems when wearing an exoskeleton in case this method is chosen to be used.

As tracking technology continues to evolve, the challenge remains achieving the best combination of precision, latency, and wearability. The ideal tracking solution for virtual reality systems using exoskeletons depends strongly on the use case, but it would likely be one that is the best compromise between precision, price, and comfort.

Prototypes and Products

Dexta Robotics has been conducting research on human-machine-interactions for virtual and mixed reality applications since 2014. They company believes that the user interface fundametally defines the user experience. Just the way motion controllers took over console controllers in the virtual reality space, the next significant step could involve the adoption of glove-based controllers over the motion controllers to achieve a more intuitive, button free and hands-like experience. Their latest model, the Dexmo Force Feedback Gloves are currently available to purchase privately for enterprises or research institutions. These gloves are meant to be used variety of applications, including VR arcades, training simulations, entertainment, academic institutions and other scientific research faculties. While a consumer edition is still under development, the current model demonstrates promising potential. The design of the Dexmo gloves relies on rigid kinematic linkages actuated by compact gear motors. The motors were specifically designed and manufactured in-house in order to provide the small form factor. The motors are capable of providing 0.5N of torque each, allowing for significant, yet safe force feedback. Additionally, the system is fully wireless, it is powered by a battery which provides

a needed energy for up to six hours. The choice of materials and in-house built parts ensures that the entire system remains lightweight, the whole glove weights under 300 grams. In order to achieve precise local position tracking, the gloves use rotational sensors, offering 11 degrees of freedom (DOM) for the movement.

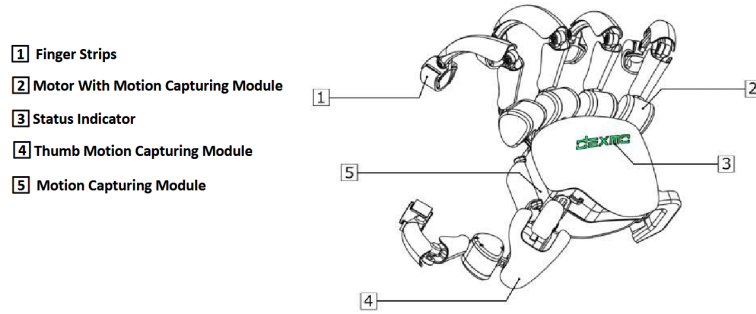


Figure 2.1: Design of Dexmo Glove

The Sensaglove Nova 2 uses a different approach for the force feedback. It utilizes magnetic friction brakes to transfer the force to the fingertips via mechanical wires. Each brake is capable of delivering up to 20 newtons of force, allowing for stronger haptic interactions. In addition to the force feedback, the glove integrates vibrotactile feedback to the fingertips using voice coil actuators. A hybrid approach is used for the local position tracking. The hybrid approach combines sensor-based finger tracking with computer vision hand tracking algorithms in order to achieve a more accurate finger tracking with more degrees of freedom.

DextrES [15] research paper proposes a more innovative yet experimental approach. They rely on flexible metal strips to generate an electrically-controlled friction force. Such design is much more compact and light as it does not rely on motors. The haptic glove that weighs only 8g is capable of generating up to 20N of holding force for each finger. Despite being lightweight, flexible and providing strong feedback, this approach has a drawback: due to the principal of breaking, the glove can only resist and stop the movement exerted by the hand, but the virtual objects can not push against the virtual hand in order to pull your physical fingers backwards.

A distinct approach to data collection for the purpose of robot arm learning from demonstration has been proposed [8]. The system has a gripper like form factor making the data collection intuitive. The glove uses displacement and inertial measurement unit sensors in order to capture position, orientation and gripper opening displacement information. However, unlike previously mentioned gloves, this exoskeleton provides no force feedback, its primary

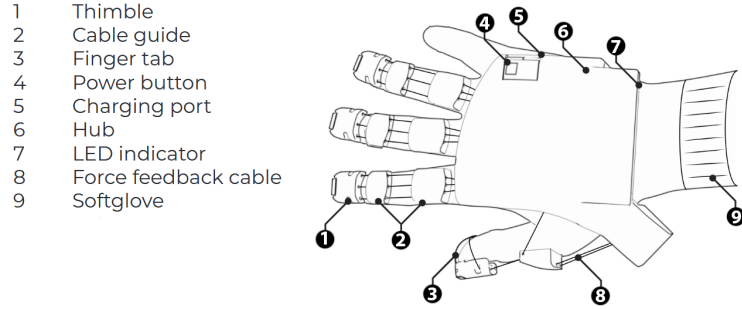


Figure 2.2: Design of Sensaglove Nova 2

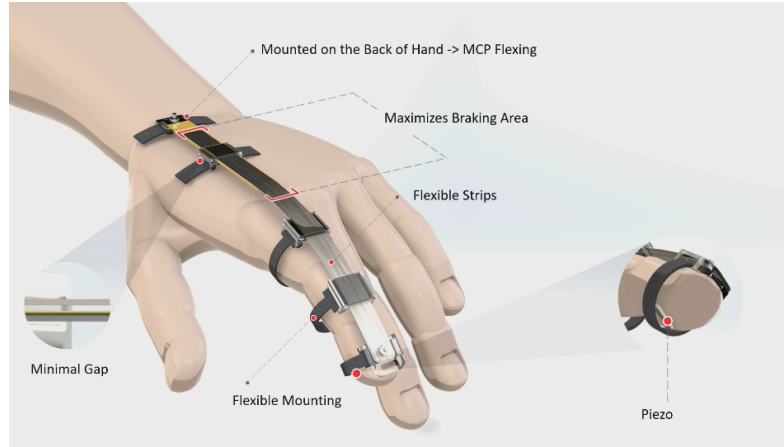


Figure 2.3: Design of DextrES

function is to interact with real objects rather than simulated ones. This research paper was included in this section for interesting gripper like form factor. The exoskeleton does not need to be a glove if the goal of interaction with the simulated virtual environment is to teach robots to perform tasks.

Interesting approaches has been proposed for variable stiffness control using the principle of jamming. This principle relies on changing the mechanical properties of materials by altering their internal friction or structure. One such implementation is HJ-Haptic [21], it uses this principle to provide stiffness feedback in bilateral teleoperation of robotic gripper. The device consists of flexible, honeycomb-structured layers that, when compressed through vacuum-induced suction, press together, increasing their internal friction, thereby altering the perceived stiffness. The stiffness increases linearly with the increase in pressure, allowing easy control of force feedback. A related approach has been

proposed in the development of a lightweight AR glove [16]. It uses 3d printed fibers in order to achieve the layer-jamming technique for force feedback. This method ensures minimal weight and flexible finger movement, making it a promising candidate for wearable haptic interfaces.

Lastly, many exoskeleton researchers opt to use series elastic actuators (SEAs). This approach allows the motor to be placed outside the exoskeleton, reducing the weight and worry of needing to fit all of the components compactly on the hand. SEAs transmit the motor torque through Bowden cables. Between the motor and the load a spring is placed, to introduce compliance and enhance safety. The transmitted force is often calculated by measuring the deflection of the springs and applying a Hooke's Law, allowing for position and torque control [25], [33].

These developments in haptic exoskeletons, gloves and various prototypes exploring different mechanisms for enhancing virtual touch experiences shows that the field is evolving rapidly. While many of these technologies remain in the experimental or enterprise-focused stages, continued research and refinement is likely to bring them closer to widespread consumer adoption.

2.2 Analysis and Design 1

A design using magnetic friction brakes is not an ideal solution as it provides feedback primarily during the squeezing of virtual object, during releasing the brakes are disengaged and no resistance is felt. This may be a good solution for VR simulations consisting of only rigid objects, but since this thesis is focused on soft objects an exoskeleton with servo motors will be used. The purpose of the first design is to get a good idea of the scope of the project, make a working prototype quickly and detect the possible issues with the project early on. The prototype, which from now on will be referred to as RoboGripVR V1, consists of two Dynamixel XM540-W270-R servo motors connected in daisy-chain using single line of RS-485 communication. The Dynamixel XM540-W270-R is a high-performance and precision servo motor from Robotis, this motor can provide 10.6 Nm of torque at 12V. The SMPS2Dynamixel is a power converter for the motors and U2D2 is a USB communication converter that enables to control and operate DYNAMIXEL motors using a personal computer (PC) Fig 2.4.

This design is not in the glove form, so it does not have sensors for global position tracking. The squeezing motion can be performed using thumb and index finger.

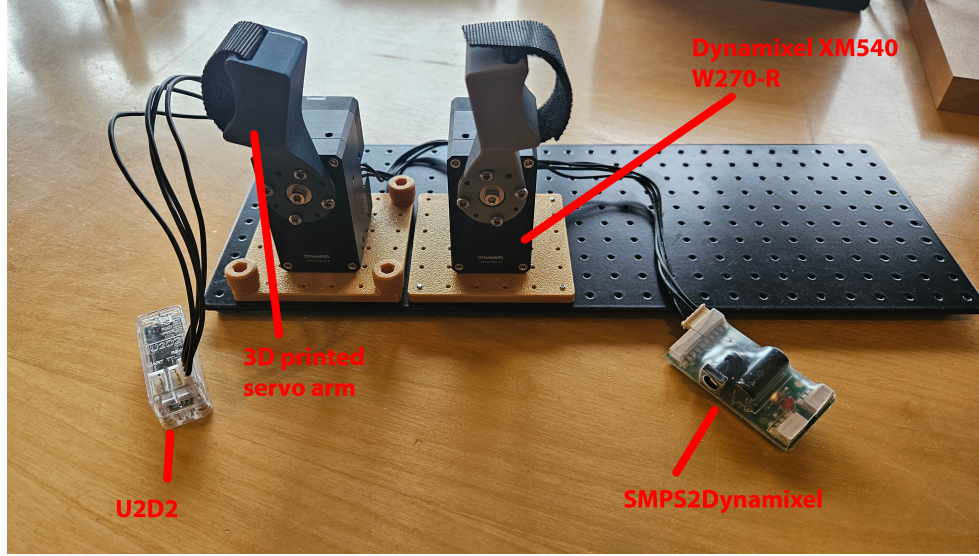


Figure 2.4: Design of RoboGripVR V1

2.3 Implementation 1

The PyBullet environment for testing this design consists of a table, soft ball and a robotiq 140 gripper Fig 2.5. The table and gripper were imported using a Unified Robot Description Format (URDF) an XML-based file format used to describe the physical and kinematic properties of a robot. The soft ball was created using open source 3D modeling software - Blender. The blender file exported as .obj provides only the surface mech. To obtain volumetric mesh needed to simulate the soft body, open source software - GMSH was used.



Figure 2.5: PyBullet environment for testing RoboGripVR V1

The gripper has stroke range of 0 to 0.127 units in the simulation. It is controlled by a single motor using position control. A function that controls the robotiq gripper based on distance instead of angle was created by linearly spacing 100 points of gripper's stroke range, measuring the angle of the motor and

the distance between the fingerpads in simulation and fitting this data using a polynomial Fig 2.6.

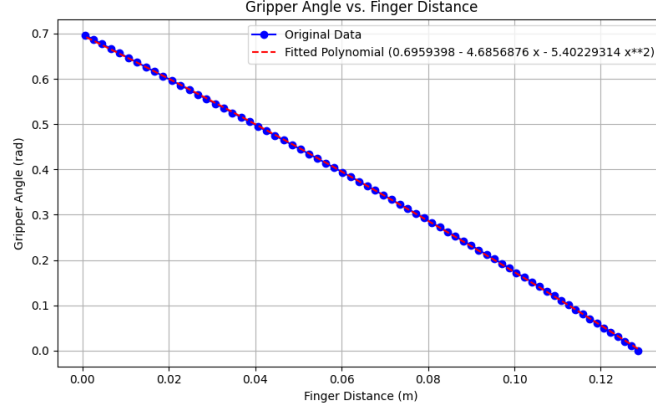


Figure 2.6: Polynomial fitting of data between gripper’s motor angle and distance between finger pads

The Dynamixel motors are controlled with the python files provided in their SDK. The distance between finger pads can be calculated using simple trigonometry Eq 2.1, Fig 2.7. This distance obtained from the motors is scaled and applied to the simulation every frame.

$$\begin{aligned} p_1 &= r_1 \cdot \cos(a_1) \\ p_2 &= r_2 \cdot \cos(a_2) \\ d_{total} &= -p_1 + d_0 + p_2 \end{aligned} \quad (2.1)$$

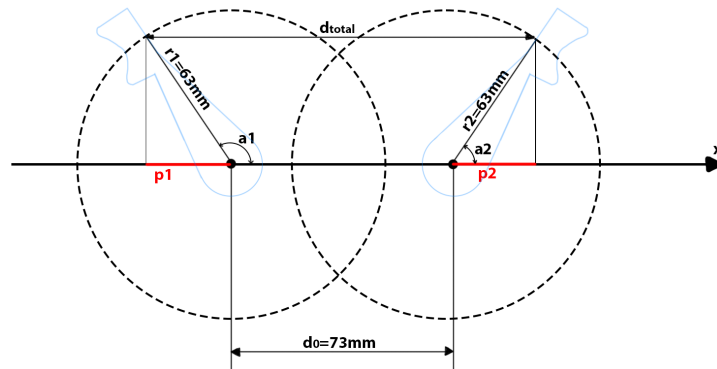


Figure 2.7: Diagram of distance calculation for RoboGripVR V1

The PyBullet’s Neo-Hookean model was used for soft-body simulation. While linear models like Hooker’s Law can be used to model small strains, materials

that undergo large deformations need to use a hyperelastic model that account for their nonlinearities. Neo-Hookean is a simple form of hyperelastic model. It is characterized by strain energy density function, which defines the energy stored in the material due to deformation. The Neo-Hookean model decouples this energy into two parts, deviatoric that handles shear deformations and volumetric part that deals with volume changes. The model is configured by material dependant Lamé parameters that describe the mechanical behavior of isotropic solids. The first Lamé parameter λ is related to the volumetric changes and the second Lamé parameter (shear modulus) μ is used to describe the material's resistance to shear deformation. Both of these parameters can be calculated [11] from material properties E (Young's modulus) and ν (Poisson's ratio) Eq 2.2, 2.3.

$$\mu = \frac{E}{2(1 + \nu)} \quad (2.2)$$

$$\lambda = \frac{E\nu}{(1 + \nu)(1 - 2\nu)} \quad (2.3)$$

A study [17] has shown that it is possible predict these parameters using data driven model to accurately simulate real world soft models in PyBullet. It also mentions that soft-bodies tend to become unstable (implode or explode) when Lamé parameters are set too low or to high. Another reason for soft-body instability is that the simulation is run in discrete time steps, the lack of fidelity can cause numerical issues during sudden application of large forces. Some solvers use adaptive time stepping to overcome this issue, but such feature is unavailable in PyBullet. It's a common practice to set PyBullet's time step to 1/250 of a second, but this value turned out to have poor stability during the experiments. A value of 1/1000 was used as it yielded much better stability. Another factor that influenced stability was the mesh itself. Multiple balls were created using icosphere mesh with varying subdivision setting Fig 2.8. Increasing subdivisions increases the resolution of the mesh, making the ball appear smoother, but it also increases the vertex count potentially making soft body simulation unstable or slower. A ball with 2 subdivisions was used as it yielded the best performance and stability.

The normal forces between soft ball and gripper pads are obtained using PyBullet's function `getContactPoints`. The reaction forces R_l and R_r in PyBullet are assumed to be distributed symmetrically and can be represented with total resultant force as a sum of individual forces at the centroid of the contact area. In order to achieve the force feedback in the exoskeleton, the reaction forces from simulation are converted to motor torques Eq 2.4

$$\begin{aligned} \tau_l &= R_l d_l \sin\theta \\ \tau_r &= R_r d_r \sin\theta \end{aligned} \quad (2.4)$$

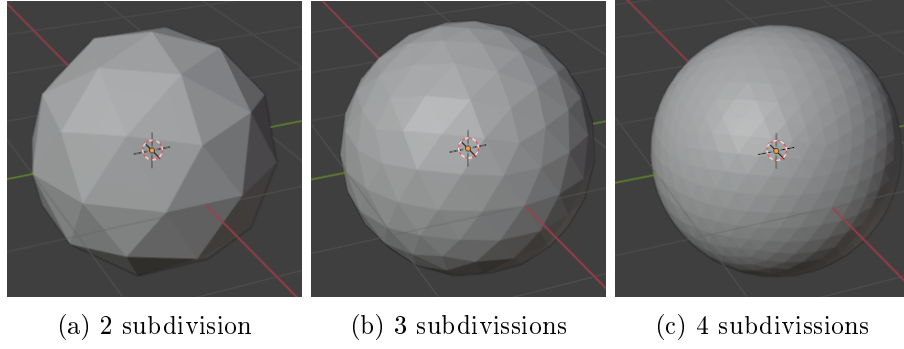


Figure 2.8: Varying subdivision setting

where:

- τ is the motor torque required to exert equivalent reaction force,
- R is the equivalent normal reaction force,
- d is the distance between the reaction force and the center of rotation,
- θ is the angle between force and the lever arm.

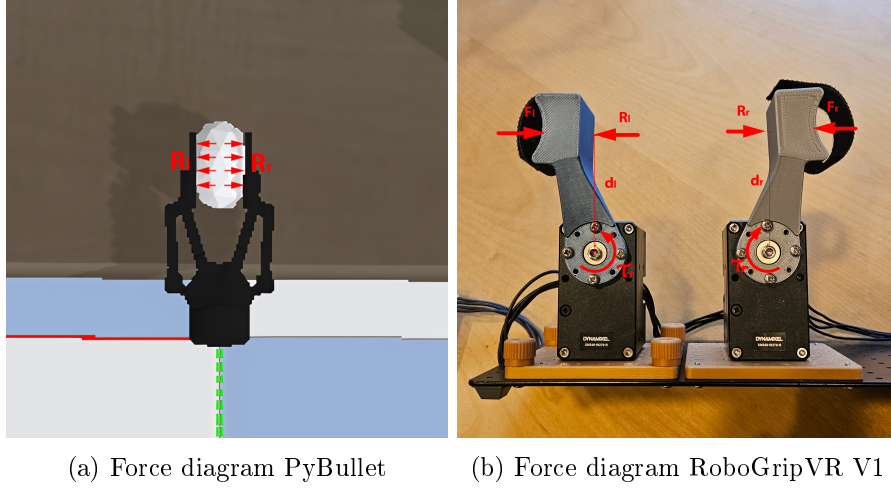


Figure 2.9: Force diagrams

XM540-W270-T/R are smart servo motors; they have several control modes available: Current Control Mode, Velocity Control Mode, Position Control Mode (0 360 [°]) and PWM Control Mode (Voltage Control Mode). The current control mode and PWM control mode could both be suitable in this scenario, so they will be tested and comparison between them will be made.

PWM control

The Pulse Width Modulation (PWM) is a low level motor control mode and is the final step in all of the control modes. It can give fine grain control over the motors if the available control modes are not satisfactory. Additionally, not all motors provide multiple control modes, it is common for direct current (DC) motors to only use PWM control as a means to change the speed of the motor. PWM signal is a square wave with the signal either being on (set to the maximum voltage) or off (zero voltage). The average voltage delivered to the motor depends on the duty cycle, a ratio of the "on" time to the total period of the signal. For the motor used $PWM \in \mathbb{Z} \cap [-885, 885]$, where 0 would be duty cycle 0%, 885 duty cycle 100% in counter clock wise (CCW) direction and -885 duty cycle 100% in clock wise (CW) direction.

$$V_{out} = V_{max} \cdot DutyCycle \quad (2.5)$$

The motor torque holds a linear relationship to the supplied current Eq 2.6, where k_t is torque constant. From the XM540-W270-T/R motor's datasheet, we can see that stall torque is 10.6 [Nm] (at 12.0 [V], 4.4 [A]). Using this information the torque constant can be calculated: $k_t = \tau/I = 10.6/4.4 = 2.409$

$$\tau = k_t \cdot I \quad (2.6)$$

The motor can measure the supplied current, thus the goal of PWM control will be to reach and maintain a goal current instead of torque as the value can be converted back to the torque later on. The PID controller diagram can be seen in Fig 2.10, the motor's servo arm is placed against a fixed support, once the PWM signal is set, the reaction force from the fixed support generate resistance that is measured by the motor in form of current.

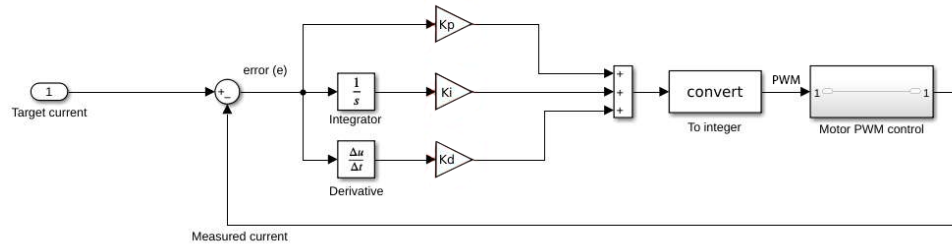


Figure 2.10: Diagram of PID controller

The K_p , K_i and K_d gains need to be tuned. When the transfer function is known, analytical methods such as root locus could be used to tune a

PID controller to meet performance requirements. However, for model-free systems, manual tuning is commonly employed, or heuristic approaches like Ziegler–Nichols can be used to obtain initial controller parameters. Manual tuning was employed, but it proved to be difficult and time consuming. It was also very easy to get stuck in a local minima as drastic changes in gains require to restart tuning process. During this trial and error phase an interesting idea occurred to me. By employing evolutionary algorithm (EA), variety of gains could be tested autonomously based on desired fitness function. When the ball in simulation is squeezed the reaction forces and thus the target currents will be changing rapidly. Intuitively, a good fitness function should encompass rapid changes. For this reason a target current is chosen as a step function Eq 2.7 and the fitness of the individuals will be a sum of absolute errors, meaning the best performing members of the population will be able to reach the target signal with minimal rise time and steady state error.

$$f(\text{iteration}) = \begin{cases} 0.2 & \text{if } 600 < \text{iteration} < 1200 \\ 0.1 & \text{otherwise, where iteration} \leq 1800 \end{cases} \quad (2.7)$$

A population of size 150 was initialized. Each member was assigned random P, I, and D values in ranges $[0, 1024]$, $[0, 512]$, and $[0, 64]$, respectively. The algorithm was trained for 20 generations with a mutation rate of 0.1. The top-performing member of the population has the PID values $K_p = 28.809$, $K_I = 503.211$, and $K_D = 0.191$ with the fitness value of 2.295. This is a significant improvement as the fitness for a manually tuned controller is 8.824.

After the training the PID controller was tested against a more varied target function that includes triangle and sinusoidal waves Fig 2.11. The controller is able to follow the target quite well, the fitness for this function is 54.8. On average in the testing function the present current is delayed by 17 iterations, which equates to 131.059ms with average iteration time being 7.709ms

The exoskeleton was also tested by squeezing the virtual ball. The sensation subjectively feels convincing, but looking at the Fig 2.12 it is evident that the significant delay in reaching target current results in a discrepancy between the forces according to the simulation and forces that are felt. Note that the controller is disengaged when the target current is zero, otherwise the PWM controller generates resistance instead of being in freewheeling condition.

Current control

This control mode allows the current to be controlled directly. Internally it works similarly to the controller we have implemented in the section above. As seen in Fig 2.13 when tested, this controller has a much better performance

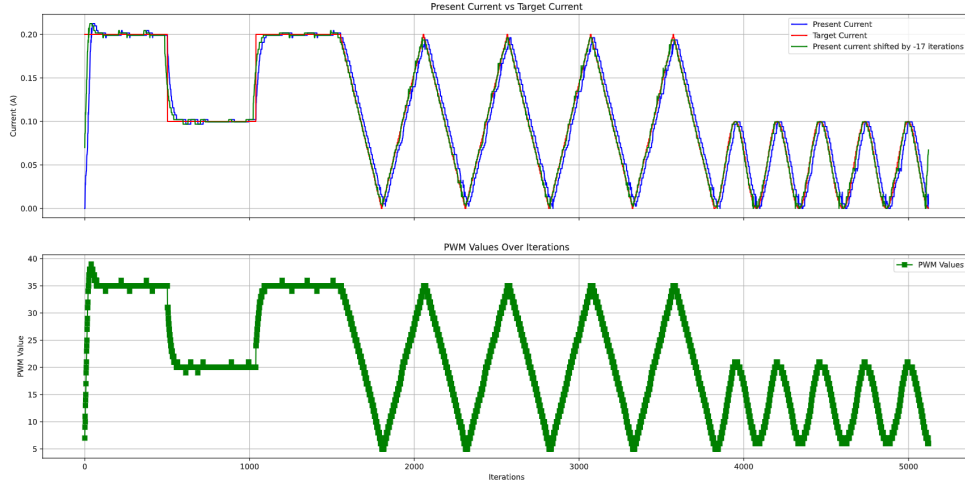


Figure 2.11: Function for testing trained PID values

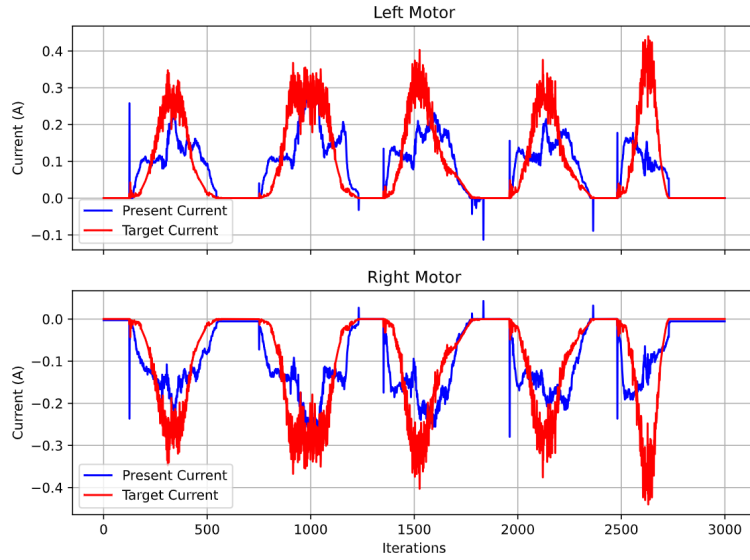
compare to our implemented PWM control, with fitness (sum of absolute errors) being 18.823 and reaching target current in less than 8ms (one iteration, exact time is unknown as this is as fast as we can measure). This performance can be attributed to Dynamixel having well-tuned PID loops optimized for their hardware and low latency feedback due to motor performing controlling internally without the need of slow communication and signal conversion to the PC.

When squeezing the virtual ball (Fig 2.14) we can see that the current is followed closely, but subjectively the experience does not feel smooth, rapid oscillations in target current occur. By taking a closer look to a single squeeze Fig 2.16a we can see how uneven the target current is. This issue could be mitigated by processing the signal using exponential smoothing Eq 2.8. A variety of different window size values have been tested and plotted 2.15. Increasing the window size provides smoother signal but adds a higher delay. For the controller a window size of 10 is used as it provides a smoother signal without much delay 2.16b. This controller still does struggle to maintain low target current when suddenly introduced with external forces from the fingers.

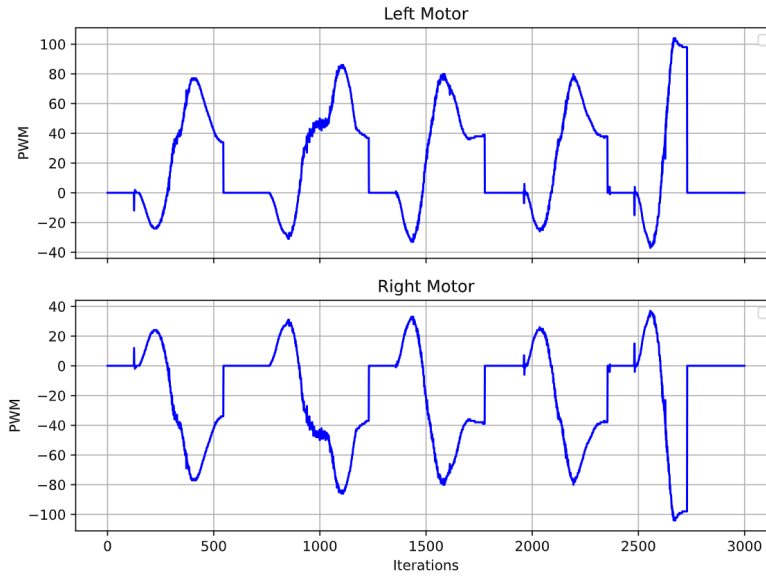
$$S_t = \alpha X_t + (1 - \alpha)S_{t-1}, \quad \text{where } \alpha = \frac{2}{K + 1} \quad (2.8)$$

where:

- S_t is the smoothed value at time t ,
- X_t is the actual value at time t ,
- α is the smoothing factor, calculated using window size K ,
- S_{t-1} is the previous smoothed value.



(a) Current



(b) PWM

Figure 2.12: Squeezing the virtual ball 5 times using PWM control

2.4 Analysis and Design 2

The purpose of a second design is to make the exoskeleton wearable. Ideally it should be easy to make, cost-effective, durable and functional.

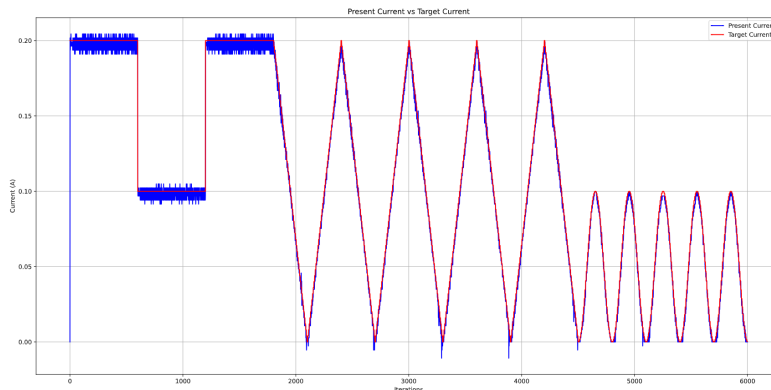


Figure 2.13: Current control test

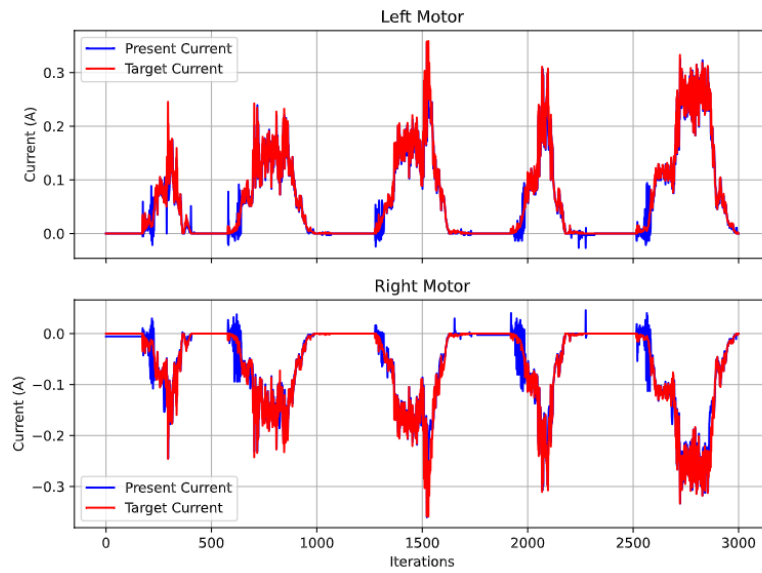


Figure 2.14: Squeezing the virtual ball 5 times using Current control

2.5 Conclusion

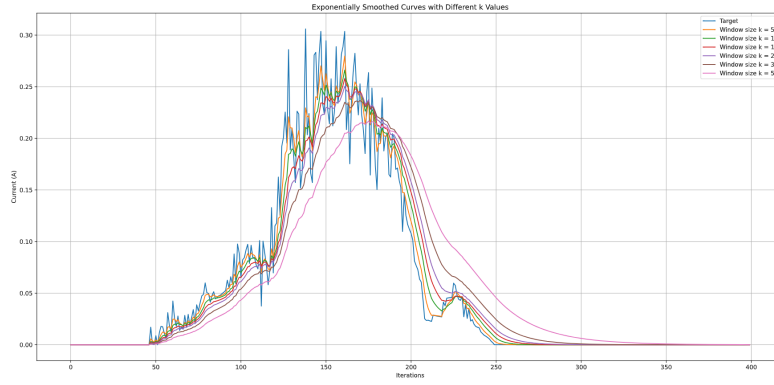
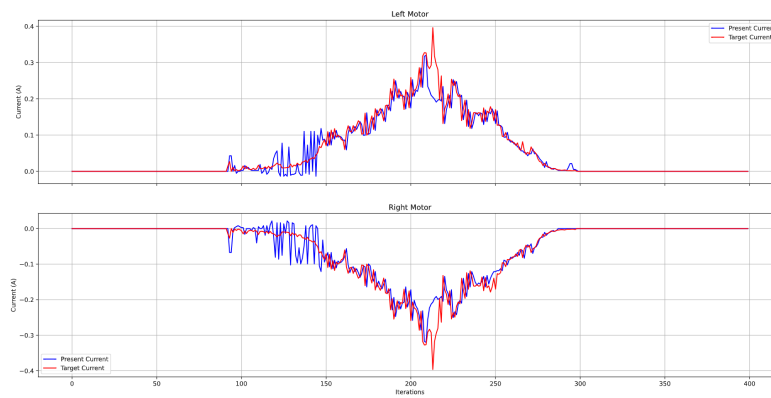


Figure 2.15: Exponential smoothing with different window sizes



(a) No target current smoothing

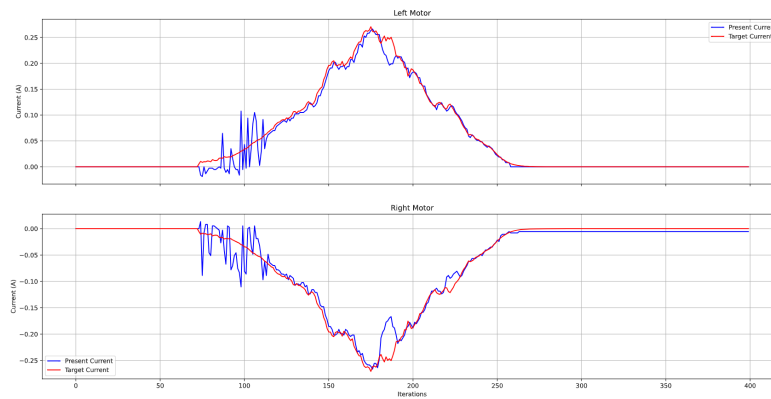
(b) Target current exponential smoothing with $K=10$

Figure 2.16: Influence of exponential smoothing when squeezing a virtual ball

3 Algorithms

In order to test if the proposed exoskeleton can aid in the training of the robots, a simulated environment with a task has been created in which an UR5 robot arm will have to pick-and-place virtual balls of varying stiffness into appropriate boxes. This task will be performed using three different methods: a classical approach, where the robot trajectory is generated from points, a reinforcement learning approach, where the robot aims to perform a task by trial and error, while learning from the past mistakes and trying to maximize a reward metric and lastly, inverse reinforcement learning, where the rewards function is learned from human demonstration instead of being specified manually.

3.1 Background

Classical Approach to Control

In 1961, Unimate, the first industrial robot, which worked on General Motors assembly line, was created. The Unimate was a hydraulically operated 5 degrees of freedom arm that performed pick-and-place tasks. The robot could be programmed by manually moving a robot arm to the a desired position and recording the position. Then the robot could replay the movement by moving to the same position. This resulted in the first of it's kind point-to-point movement which is still a standard feature in many robotic arms today. Such control method is quite simple, but it does require manual programming, meaning it is most suited for simple, repetitive tasks in well known environments. Besides that, the point-to-point movement has two obvious disadvantages. Firstly, the robot movement does not appear smooth, secondly, there must be no obstacles between the points. These problems have been addressed in modern robotic control. The most simple way to move between two points is use linear interpolation. Similarly, orientation can be interpolated, depending on the representation chosen. For example, if orientation is represented as quaternions, a spherical linear interpolation (SLERP) [36] can be used. This is because quaternion is a rotation representation on 4D unit sphere and in order to move between two points a sphere, the movement should be done on a following a geodesic. In theory linear interpolation results in constant velocity motion, but in practice this means that at the beginning of the motion robot will need an infinite acceleration to achieve desired velocity instantly and at the end of the movement, it will need infinite deceleration to stop immediately. Objects with mass resist changes in motion due to inertia. The higher the object's mass is, the more force will be needed to accelerate it. The resis-

tance to rotation also depends on the shape of the object, which is described by the moment of inertia. As robot is a system constrained by physical limitations, it will not experience infinite acceleration, but the motors will likely apply the maximum torque they are capable of during the brief moments at the beginning and the end of the motion. This increases wear and tear of the motor and other components, leading to the decreased lifespan of the robot. This issue can be mitigated by using a parabolic trajectory instead. Assuming the start velocity of zero and end velocity of zero, the parabolic trajectory will have a velocity profile that linearly increases until it reaches the half point of the trajectory and linearly decreases until it reaches zero again at the end of the trajectory. Consequently, the acceleration during first and second half of the movement will have much lower maximum value and will be constant, non-zero during majority of the motion. This leads to least wear and tear, but also slower movement. A balance between trajectory duration and maximum acceleration can be achieved by merging these two methods. Linear trajectories with parabolic blends have a linear part in the middle and parabolic at the start and end of the interpolation.

When moving between two points, it is not always clear if the robot will be able to move between them without collision. This is especially true with high degree of freedom robotic arms as even if the path for the end effector is clear, other robot parts may be in collision due to the configuration of the robot. It is a difficult problem to address efficiently due to high dimensionality of the problem. Many modern robotics incorporate Rapidly-exploring Random Tree (RRT) [22] or RRT* [18] into their path planning algorithms to address this problem. RRT explores robot's configuration space, which is a high dimensional space that represents every possible configuration the robot can be in. C-Space can be divided into two sets, C-Free a space with valid robot configurations and C-Obs a space where robot configuration would be in a collision.

$$\begin{aligned} C &= C_{\text{free}} \cup C_{\text{obs}} \\ C_{\text{free}} \cap C_{\text{obs}} &= \emptyset \end{aligned}$$

RRT can find a collision free path between the two configurations in C-Free space if one exists. The algorithm builds a tree like structure by selecting a random point in a C-Free space, connecting it to the closest node in a tree (which is the initial configuration during the first iteration), and growing a branch from that tree towards the random point until it or maximum distance is reached. That will be the new location for the node to be added into the tree if the path between the node and the closest points in the tree is obstacle free. Then a new random point in a C-Free space is selected and this process continues until a tree grows large enough so a path that connects the two points in configuration space is found. This method however does not yield shortest path, RRT*

addresses this issue by doing additional processing before placing a node in a tree. Within a specified search radius around the new point, RRT* checks if the nodes can be reconnected in a way that yields a shorter path. It has been proved that when the time goes to infinity, RRT* will yield the shortest path between two points by continuously rewiring the tree.

The linear interpolation between points with or without the use of parabolic blends, results in a robot moving in a straight line between the points. A more efficient and smooth path can be achieved by using splines. Splines ensure a smooth transition between path segments, avoids abrupt changes in velocity and acceleration and minimizes jerk. An easy way to implement splines in a path with many points is to use B-splines. This approach is recommended only for the parts in the trajectory that do not need to go through the point exactly, as B-splines are just an approximation of the path, unlike interpolation methods that would go exactly through the point. B-splines are implemented by dividing each segment between points to the thirds and connecting a second third of the first segment to the first third of the second segment with a line. The center of this line will act as an anchor point for fitting a Bezier curve. The B-splines are amazing not only because they are easy to implement, but because they have C^2 continuity and local control. Local control means that moving a single point in the trajectory will affect only that local portion of the trajectory, keeping the rest of the trajectory unaffected. C^2 continuity (second-order continuity) ensures that the position, velocity and acceleration in the trajectory are smooth. C^0 position continuity ensures that the end of one segment matches the start of another. C^1 velocity continuity ensures that the first derivatives at the end of the one segment matches the start of another. And C^2 acceleration continuity ensures that the second derivative matches at the junction points between the segments. It is possible to achieve even higher order continuity, for example quintic splines can achieve a C^3 jerk continuity or even higher if higher degree polynomial is used to interpolate between the points. Quintic splines also go through the points unlike the B-splines. This could be desirable in cases where the points are defined manually. Quintic splines do come with several disadvantages, such as, lack of local control and computational complexity, so the choice of splines used would depend on the use case scenario.

Neural Networks

Neural network is a mathematical model inspired by biological neural network - the human brain. Neural networks are used in machine learning to achieve tasks that would be difficult or impossible by writing an algorithm for the task manually. The feedforward neural network [24] was among the first types of the neural networks. The information in such a network only flows forward from input to the output, which is very suitable for classification tasks. Multilayer

Perceptron (MLP) is a type of feedforward neural network that is divided into one dimensional layers, input layer, one or more hidden layers and an output layer. Each of these layers have one or more neurons. A neuron can be imagined as a place where a single, usually floating point value is stored. A simple and common approach to connect these layers is to use full connections, where each neuron is connected to every neuron in the previous layer. Modern networks can have millions and even billions of neurons, in such cases techniques where weak connections are cut are implemented to optimize the networks. The value that the neuron receives is calculated by multiplying the value of the previous neuron by the strength of the connection (weight) and addition of a bias. This process is repeated for all connections and summing up these values. As the neuron multiplication by a weight and addition of a bias is a linear function, the neurons need an additional activation function in order to learn non-linear patterns from the data. A common and popular activation function is sigmoid $f(x) = \frac{1}{1+e^{-x}}$, which also clamps the neuron values between -1 and 1. In big networks however sigmoid adds a lot of additional computation, often ReLU [30] is preferred. It is calculated simply as $f(x) = \max(0, x)$. Turns out clamping negative values is enough to provide the desired nonlinearity to the network. ReLU however sometimes presents another problem, if the network is poorly initialized with a lot of negative weights and biases, the learning rate is too high or the data often leads negative neuron activations, the neurons stop learning. This is because after negative neuron value passes through the ReLU function it becomes zero. This is called a dying ReLU problem where too many neurons output zero for all inputs, effectively making them inactive and that prevents them from learning. Leaky ReLU addresses this problem, instead of clamping negative values to zero, they are multiplied by a small constant e.g. $\alpha = 0.01$, this neurons from dying. For multi-class classification problem, the logits (raw outputs of the last layer before applying the activation function) are passed through a softmax function $\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$, this outputs the probabilities of input belonging to each class, the highest probability is often chosen as a network prediction. The sum of probabilities of input belonging to each class is 1. The data for multi-class classification is usually one-hot encoded, where the probability of it belonging to the true class is 1 and 0 for the rest. The loss for such problem is calculated using cross-entropy

$$\mathcal{L} = - \sum_i y_i \log(\hat{y}_i)$$

where:

- y_i is the true label (usually one-hot encoded, meaning one class is labeled as 1, others as 0)
- $\hat{y}_i$ is the predicted probability for class i from softmax

The cross-entropy measures the difference between two probability distributions, leading to stronger gradients than using mean square error. For one-hot encoded dataset, the cross entropy simplifies to $\mathcal{L} = -\log(\hat{y}_c)$ where $\hat{y}_c$ is the predicted probability of the true class. The goal of neural network training process is to minimize this loss. This is done through a process called backpropagation. The weights and biases need to be adjusted in a way that leads to lower loss. In order to know how the changes in weights and biases affect the loss a gradient can be used. During the backpropagation the gradients are calculated for each weight and bias by the use of gradient chain rule. This way a simple way to minimize the loss is achieved, simply taking a step for each weight and bias in the negative gradient direction will result in lower loss until either a local or global minima is reached. Often when the network is big and there is a lot of data, it is unpractical to use all of the data for each pass as it is too resource intensive, often the data is split into smaller batches before being passed through the network. Another way a neural network training process is sped up is instead of using the vanilla gradient descent, a modern optimizer like adam is often employed. Adam (Adaptive Momentum Estimation) improves the gradient descent with two key ideas. Firstly, it incorporates momentum by using the mean of past gradients. This helps optimizer move in a constant direction and to reduce oscillations. Secondly, it optimizes the step size for each parameter individually based on past updates. This results in faster convergence, less sensitivity to noisy gradients (which can occur in mini-batch training) and less sensitivity to hyperparameter tuning.

Over the years, the feedforward neural network evolved into different architectures that are more suitable for specific tasks. Recurrent Neural Networks (RNNs) [35] build upon a feedforward neural network by introducing loops in the network, a hidden state that carries information from the previous inputs. This makes RNNs ideal for tasks where order matters, such as language modeling or speech recognition. The hidden states however can not retain information over long period of time, for that an improvement was made with Long Short-Term Memory architecture. It uses cell state to carry information across time steps and gates that regulate what information should be store, what should be forgotten and what should be passed as an output. For image recognition and feature extraction tasks Convolutional Neural Networks (CNNs) [19] are often employed. Classical image processing tasks, such as blurring, sharpening and edge detection are often achieved by sweeping a square 3x3 or bigger kernel through the entire image and applying a convolution operation between kernel and each area in order to produce a new image pixel. Instead of using predefined kernel values, convolutional layers in CNNs learn these values during backpropagation. Image passed through convolution layers is called a feature map. A single image often produces many feature maps. Parameters like stride, padding, kernel size and pooling layers affect the size of the feature map, but it is often smaller than the image itself, leading to a

dense representation of the most important features. These feature maps then are flattened to one dimension and concatenated together before being passed through a feedforward neural network. CNNs themselves can only be around 15 layers deep before experiencing vanishing gradient problem. It has been found that deeper network can result in better feature extraction, a breakthrough was achieved with residual neural network (ResNet) [14] a type of CNN architecture that overcame the vanishing gradient problem. The ResNet layers can be 50 or even a 100 layers deep, making them the choice in many pre-trained backbones. For data compression, denoising and anomaly detection autoencoders are often used. They are often a fully connected networks with a bottleneck in the middle. This bottleneck is a lower-dimension space called latent space. It is the part of the network that captures the essential features of the input data.

3D Camera Perception

Reinforcement Learning

Reinforcement learning (RL) is a subset of machine learning. There are three main types of machine learning, supervised learning, unsupervised learning and reinforcement learning. Supervised learning has labeled data, a model finds patterns that relate between labeled input-output data and once trained it can use those patterns to predict outcomes not yet known. Unsupervised learning has no labeled data or human feedback, instead it finds hidden structure in the data itself and can be used to cluster it. RL learns through trial and error by interacting with the environment, receiving feedback in the form of reward or punishment 3.1. It is ideal for tasks that require sequential decision making, such as playing games, navigating self-driving cars or controlling robots.

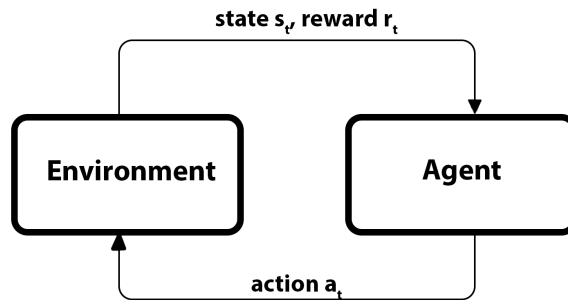


Figure 3.1: Reinforcement learning model

In order to understand RL, it is important to understand the key concepts and the terminology:

Agent - the agent is the decision maker. In the context of a game it would be a player and in the context of this thesis it will be a robot arm.

Environment - it is a place in which the agent resides in. It encompasses everything that the agent perceives and acts upon.

Action - actions are choices available to the agent in a given state. The actions can be discrete, where the number of moves the agent can perform is finite, for example two actions could be open or close the robot gripper. The actions can also be continuous, they are often represented by a real number or a vector of real numbers. Robot's actions are often continuous, such as opening a gripper to a certain distance, or moving robot's joint to a certain angle. Reinforcement learning methods and approaches often differ depending if the actions are discrete or continuous.

Action space - set of all valid actions in a given environment.

Reward - the feedback from the environment. A numerical value associated with how good the agent's action was. A positive value represents a reward and a negative value represents a punishment. It depends on the state, action and the next state of the environment $r_t = R(s_t, a_t, s_{t+1})$. Although frequently the reward dependance on inputs is simplified only to state-action pair $r_t = R(s_t, a_t)$ or even the current state only $r_t = R(s_t)$. The agent's goal is to maximize the cumulative reward. Designing a reward function that aligns with the desired behavior of the agent is a challenging task often requiring lots of trial and error by tuning and retraining the agent. Another difficulty are sparse rewards, for example the winner in the game of chess is only decided after the game has ended, it is difficult for the agent to know which of the moves led the agent to the victory or loss. Sparse rewards make the agent learn at a much slower rate leading to extensive training times.

Discount factor - $\gamma \in [0, 1]$ is a parameter that discounts the future rewards. Often instead of weighting all of the rewards equally, more weight is given to short term rewards. Total cumulative reward over trajectory τ with discount factor is calculated as $R(\tau) = \sum_{t=0}^{\infty} \gamma^t r_t$. If the discount factor is high (e.g. 0.9 - 0.99) the agent will try to learn long-term strategies and if it is low, it will focus on quick gains.

Policy - policy is what defines the agent's strategy. It is a mapping between states and actions. There are two types of policies, deterministic and stochastic. Agent following a deterministic policy will always choose the same action given the same state. On the other hand, agent following a stochastic policy will have a probability distribution over the available actions.

Value function - represents an expected cumulative reward for a certain state or state-action pair if the agent were to act according to the same policy forever after. It shows how good a particular state or action is. State-value function $V(s)$: represents expected reward starting in a state s . Action-value function

$Q(s, a)$: represents expected reward for taking action a in state s and following the policy thereafter. The action-value function is also known as Q-function.

Exploration vs. Exploitation - is a key challenge in RL on how to balance between exploration and exploitation. Exploitation means the agent chooses the best known action in order to maximize it's reward. Exploration means agent tries less known action in order to discover new, potentially better strategies. There are several exploration vs exploitation balancing strategies, one that is simple and well known is called $\epsilon - Greedy$.

During beginning of the training $\epsilon \in [0, 1]$ is initialized to a high value (e.g. 1), making the agent perform mostly random actions and as training continues epsilon decreases, making the robot prefer to exploit the policy Eq. 3.1.

$$a = \begin{cases} \text{random action,} & \text{with probability } \epsilon \\ \arg \max_a Q(s, a), & \text{with probability } 1 - \epsilon \end{cases} \quad (3.1)$$

On-Policy and Off-Polict learning. These are two ways the policies can be categorized. On-Policy learning refers to a process when the agent continuously updates the same policy it uses to interact with the environment. On-Policy learning often results in more stable, but much slower training compared to Off-Policy training. Off-Policy training refers to a process when agent learns from experiences collected using different policy than the one it is currently improving. The agent saves a tuple $(s_t, a_t, r_t, s_{t+1}, done)$ to a replay buffer after each action it takes. during the training loop a number of random experiences is sampled from the replay buffer and used to update agent's policy. This parallelism of using many experiences at each iteration loop allow's the agent to learn quickly, but the learning may be less stable.

Model-Based and Model-Free Learning. Approaches to RL differ depending if we have a model of the environment (transition probabilities $P(s_{t+1}|s, a)$ and rewards r_t). Tasks where the model is known are less common and usually applicable to smaller problems. The Model-Free learning is a more general case, it is more suitable for large state spaces and continuous action spaces. In this case the agent learns purely through trial and error.

Many RL problems are modeled as Markov Decision Process (MDP). Problems that are modeled as MDP's assume a Markov Property that probability of transitioning to the next state s_{t+1} depends only on the current state-action pair and not the past history of state-action pairs.

$$P(s_{t+1}|s, a, s_{t-1}, a_{t-1}, s_{t-2}, a_{t-2}, \dots, s_0, a_0) = P(s_{t+1}|s, a) \quad (3.2)$$

If the environment is only partially observable or it depends on the past states, problem can not be modeled as MDP. MDP framework models the environment as tuple (S, A, P, R, γ) :

- S Set of all states
- A Set of all actions
- $P(s_{t+1}|s, a)$ Probability of transitioning to the next state s_{t+1} given state s and action a pair
- $R(s, a, s_{t+1})$ Reward
- γ Discount factor

This framework formalizes RL problems. The value functions for RL problem defined by MDP can be determined with the use of recursive formula called Bellman equation. The following are the Bellman equations for the On-Policy value functions:

$$V^\pi(s) = \mathbb{E}_\pi [r_{t+1} + \gamma V^\pi(S_{t+1}) \mid S_t = s] \quad (3.3)$$

$$Q^\pi(s, a) = \mathbb{E}_\pi [r_{t+1} + \gamma Q^\pi(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a] \quad (3.4)$$

These are used when evaluating a specific policy π . The actions are sampled from the policy $a \sim \pi(\cdot \mid s)$ and future states from the probability distribution $s_{t+1} \sim P(\cdot \mid s, a)$. The Bellman equations for the optimal policy are:

$$V^*(s) = \max_a \mathbb{E} [r_{t+1} + \gamma V^*(S_{t+1}) \mid S_t = s, A_t = a] \quad (3.5)$$

$$Q^*(s, a) = \mathbb{E} \left[r_{t+1} + \gamma \max_{a'} Q^*(S_{t+1}, a') \mid S_t = s, A_t = a \right] \quad (3.6)$$

These describe the best possible values if the agent always acts optimally. The $\max_a$ and $\max_{a'}$ denote that the agent always picks the best action that maximizes the expected future rewards. From these equations value function update methods can be derived. The On-Policy method SARSA has the following Q-function update equation:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha (r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)) \quad (3.7)$$

$\alpha \in [0, 1]$ is a learning rate. The Off-Policy method Q-Learning has the following Q-function update equation:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left(r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right) \quad (3.8)$$

Learning from Demonstration

In the previous section reinforcement learning was discussed. It is an interesting area of machine learning that generalizes to many applications requiring decision making. RL depends on agent maximizing reward function, more often than not the reward function is not clear and manually designing a reward function that satisfies the desired behaviour can be extremely challenging. A feasible solution to this problem is Learning from Demonstration (LfD), also known as imitation learning. In LfD, instead of specifying reward function, expert provides a set of demonstrations on how the task should be accomplished and the agent tries to learn the policy that best matches what the expert attempted to do. The environment in imitation learning, as well as in majority in RL tasks, is defined as a Markov Decision Process. The environment has a set of states and actions, transition probabilities between the states and an unknown reward function. The expert demonstrations, also known as trajectories, are defined as an array of state action pairs.

$$\tau = (s_0, a_0, s_1, a_1, \dots) \quad (3.9)$$

The expert demonstrations are considered to be coming from an optimal policy π^* . There are a few methods with which LfD can be achieved. In this section two common ones: behaviour cloning (BC) and inverse reinforcement learning (IRL) will be discussed.

Behaviour Cloning

Idea for behaviour cloning [2] relatively straightforward. Given expert trajectories, learn a mapping from states to actions using supervised learning. A commonly used example of behaviour cloning is an radio controlled (RC) car equipped with sensors and controller on a race track. A human expert would drive around the race track recording the state (camera feedback, speed) and actions (steering, acceleration). The state would be passed through a neural network (CNN if camera is used) and the network would attempt to minimize the loss between predicted actions and the action of the expert. Behaviour cloning is simple and efficient method, suited for tasks that do not require long-term planning and in which expert trajectories cover the whole state space. On the other hand, BC has severe limitations, it can only get as good as the expert, the agent can only perform well in situations it has seen before and the generalization is poor. Often the agent does not match expert trajectory perfectly, leading to accumulation of error over time, once the error grows large, the agent can find itself in a situation it has never seen before, this will lead to failure as the agent will not know what to do in novel states.

The paper "ChauffeurNet: Learning to Drive by Imitating the Best and Synthesizing the Worst" [3] proposes that the network is trained not only the

expert trajectories, but the dataset be augmented with simulated uncommon scenarios, such as collisions and off-road events in the case of training a car to drive autonomously. Another interesting proposition [12] is the Cycle-of-Learning (CoL) framework, where behaviour cloning and reinforcement learning are combined, by pre-training the agent from expert demonstrations and fine tuning it through RL.

Inverse Reinforcement Learning

The inverse reinforcement learning takes a different approach to imitation learning. The main idea of this method is to learn the reward function, instead of the policy directly. This learned reward function then can be used to train the agent through the reinforcement learning.

Algorithm 1 Inverse Reinforcement Learning (IRL)

- 1: **Input:** Expert demonstrations $D = \{\tau_1, \tau_2, \dots, \tau_n\}$
 - 2: **repeat**
 - 3: Learn reward function $r(s, a)$
 - 4: Given reward function r find policy π using the reinforcement learning approach
 - 5: Compare π with expert policy π_E
 - 6: **until** Policy is satisfactory
-

In the case we have a model of our system, the rewards are assumed to be linear. Then during each iteration of IRL, a full RL problem is solved using approaches, such as dynamic programming (DP). In Model-Free approach the reward function is often non-linear (neural network) and transition probabilities are unknown, in this case the reward function is continuously updated using a short-horizon updates (e.g. 1-step of RL) instead of running the full RL.

The inverse reinforcement learning has very good generalization, meaning, unlike the behaviour cloning, IRL can better deal with scenarios that were not explicitly shown by the expert as the model learns the goal rather than state-action mappings. Sometimes the policy trained with IRL can even outperform the expert. It is a good choice for more complex scenarios requiring long-term planning or decision making. On the other hand, the IRL has problems of its own. The biggest is ambiguity in reward function. There can be many reward functions that explain the behaviour of the expert. Different approaches have been proposed on which reward function to choose.

Maximum Margin Inverse Reinforcement Learning (MM-IRL) [32] envisions the problem as structured prediction over the space of policies. The goal of MM-IRL is to learn a mapping from features to costs. The features are problem

specific, for example in robot pick-and-place task, they could be: time taken to achieve a task, applied grasping force, smoothness of motion, etc. The cost is just a negative reward function. The cost function is assumed to be linear in the features.

$$R(s, a) = -C(s, a) \quad (3.10)$$

$$C(s, a) = w^T \phi(s, a) \quad (3.11)$$

where:

- $C(s, a)$ is the cost function,
- w is the weight vector that must be learned,
- $\phi(s, a)$ are the features describing state-action pairs,

Knowing the cost function is akin to knowing the reward function and thus the policy can be learned. The problem can be viewed as constrained optimization problem, where the goal is to find the set of weights that explain the expert trajectory. More specifically, the cost of expert trajectory has to be smaller or equal to cost of all the possible trajectories in MDP. This optimization problem can not be solved by simply passing it to a linear solver as the case of weight vector equal zero would satisfy the constraint. In order to avoid trivial solutions the margin term is introduced, the cost of human trajectories has to be smaller than the cost of the remaining trajectories by a margin. In order to account for cases where the set of weight is very large (and thus the margin term becomes small) additional constraint is added that maximizes the margin.

Maximum Entropy Inverse Reinforcement Learning [42] chooses the reward function that induces a trajectory distribution with maximum entropy while matching the expert's expected feature counts. The idea is built upon a moment matching principle. This principle states that if the expected values of the features between the learner and the expert match, then the expert cost will be equal to the learner cost and the behaviour of the agent will result in a similar behavior to the expert, without explicitly observing the expert's actions.

$$\mathbb{E}_{\pi_{\text{expert}}}[\phi(s, a)] = \mathbb{E}_{\pi_{\text{learner}}}[\phi(s, a)] \quad (3.12)$$

If we plot the feature counts of the expert as a histogram, there are many potential distributions that matches the average feature counts. The objective function that determines which distribution best matches features counts is chosen as the one that has maximum entropy. The idea behind maximum entropy is to choose a distribution which is least comital as this will result in the most unbiased distribution over feature counts.

3.2 Analysis and Design

3.3 Implementation

3.4 Conclusion

4 Discussion

Summing up all the main parts 1 to n conclusions.

5 Conclusion

Overall our project was a success.

A Appendix

We include here the API documentation for our library.

Bibliography

- [1] Angel Ayala, Francisco Cruz, Diego Campos, Rodrigo Rubio, Bruno J. T. Fernandes, and Richard Dazeley. A comparison of humanoid robot simulators: A quantitative approach. *CoRR*, abs/2008.04627, 2020.
- [2] Michael Bain and Claude Sammut. A framework for behavioural cloning. In *Machine intelligence 15*, pages 103–129, 1995.
- [3] Mayank Bansal, Alex Krizhevsky, and Abhijit Ogale. Chauffeurnet: Learning to drive by imitating the best and synthesizing the worst. *arXiv preprint arXiv:1812.03079*, 2018.
- [4] Murray Campbell, A. Joseph Hoane, and Feng hsiung Hsu. Deep blue. *Artificial Intelligence*, 134(1):57–83, 2002.
- [5] Inrak Choi, Heather Culbertson, Mark R. Miller, Alex Olwal, and Sean Follmer. Grability: A wearable haptic interface for simulating weight and grasping in virtual reality. In *Proceedings of the 30th Annual ACM Symposium on User Interface Software and Technology*, UIST ’17, page 119–130, New York, NY, USA, 2017. Association for Computing Machinery.
- [6] Inrak Choi, Yiwei Zhao, Eric J. Gonzalez, and Sean Follmer. Augmenting perceived softness of haptic proxy objects through transient vibration and visuo-haptic illusion in virtual reality. *IEEE Transactions on Visualization and Computer Graphics*, 27(12):4387–4400, 2021.
- [7] Jack Collins, Shelvin Chand, Anthony Vanderkop, and Gerard Howard. A review of physics simulators for robotic applications. *IEEE Access*, PP:1–1, 03 2021.
- [8] Hengtai Dai, Zhenyu Lu, Mengyuan He, and Chenguang Yang. A gripper-like exoskeleton design for robot grasping demonstration. In *Actuators*, volume 12, page 39. MDPI, 2023.
- [9] DeepMind. Open-sourcing mujoco. <https://deepmind.google/discover/blog/open-sourcing-mujoco/>, May 2022.
- [10] Liufeng Fan, Zhan Zhang, Biao Zhu, Decheng Zuo, Xintong Yu, and Yiwei Wang. Smart-data-glove-based gesture recognition for amphibious communication. *Micromachines*, 14(11), 2023.
- [11] FEBio. Febio user manual - neo-hookean model. <https://help.febio.org/docs/FEBioUser-3-7/UM37-4.1.4.17.html>, 2023.

- [12] Vinicius G. Goecks, Gregory M. Gremillion, Vernon J. Lawhern, John Valasek, and Nicholas R. Waytowich. Integrating behavior cloning and reinforcement learning for improved performance in dense and sparse reward environments, 2020.
- [13] X. Guo, Y. Zhang, W. Wei, W. Xu, and D. Wang. Thermaltex: A two-modal tactile display for delivering surface texture and thermal information. In *Haptics: Science, Technology, Applications: 12th International Conference, EuroHaptics 2020, Leiden, The Netherlands, September 6–9, 2020, Proceedings*, page 288–296, Berlin, Heidelberg, 2020. Springer-Verlag.
- [14] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
- [15] Ronan Hinchet, Velko Vechev, Herbert Shea, and Otmar Hilliges. Dextres: Wearable haptic feedback for grasping in vr via a thin form-factor electrostatic brake. In *Proceedings of the 31st Annual ACM Symposium on User Interface Software and Technology*, UIST ’18, page 901–912, New York, NY, USA, 2018. Association for Computing Machinery.
- [16] Saurabh Jadhav, Mohamad Majit, Benjamin Shih, Jürgen Schulze, and Michael Tolley. Variable stiffness devices using fiber jamming for application in soft robotics and wearable haptics. *Soft Robotics*, 9, 02 2021.
- [17] Fan-Chien Kao, Zhi-Ren Chen, Chung-Shan Shih, Shao-Huang Lu, and Pei-Chun Lin. Bridging mechanical behavior differences of deformable soft objects in simulation and experiments using a data-driven model. In *2024 IEEE International Conference on Advanced Intelligent Mechatronics (AIM)*, pages 1297–1302, 2024.
- [18] Sertac Karaman and Emilio Frazzoli. Sampling-based algorithms for optimal motion planning. *The international journal of robotics research*, 30(7):846–894, 2011.
- [19] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 25, 2012.
- [20] Vikash Kumar and Emanuel Todorov. Mujoco haptix: A virtual reality system for hand manipulation. In *2015 IEEE-RAS 15th International Conference on Humanoid Robots (Humanoids)*, pages 657–663, 2015.
- [21] Thomas Kwok, Bohan Zhang, and Wai Chow. A wearable stiffness-rendering haptic device with a honeycomb jamming mechanism for bilateral teleoperation. *Machines*, 13:27, 01 2025.

- [22] Steven LaValle. Rapidly-exploring random trees: A new tool for path planning. *Research Report 9811*, 1998.
- [23] Bor-Shing Lin, I-Jung Lee, Pei-Ying Chiang, Shih-Yuan Huang, and Chih-Wei Peng. A modular data glove system for finger and hand motion capture based on inertial sensors. *Journal of Medical and Biological Engineering*, 39, 06 2018.
- [24] Richard P Lippmann. An introduction to computing with neural nets. *ACM SIGARCH Computer Architecture News*, 16(1):7–25, 1988.
- [25] Dario Marconi, Andrea Baldoni, Zach McKinney, Marco Cempini, Simona Crea, and Nicola Vitiello. A novel hand exoskeleton with series elastic actuation for modulated torque transfer. *Mechatronics*, 61:69–82, 2019.
- [26] Riccardo Mengacci, Giacomo Zambella, Giorgio Grioli, Domenico Caporale, Manuel G. Catalano, and Antonio Bicchi. An open-source ros-gazebo toolbox for simulating robots with compliant actuators. *Frontiers in Robotics and AI*, 8:713083, 2021. Published 2021 Aug 11.
- [27] Olivier Michel. Cyberbotics ltd. webotsTM: professional mobile robot simulation. *International Journal of Advanced Robotic Systems*, 1(1):5, 2004.
- [28] Kouta Minamizawa, Souichiro Fukamachi, Hiroyuki Kajimoto, Naoki Kawakami, and Susumu Tachi. Gravity grabber: wearable haptic display to present virtual mass sensation. In *ACM SIGGRAPH 2007 Emerging Technologies*, SIGGRAPH '07, page 8–es, New York, NY, USA, 2007. Association for Computing Machinery.
- [29] Gordon E Moore. Cramming more components onto integrated circuits. *Proceedings of the IEEE*, 86(1):82–85, 1998.
- [30] Vinod Nair and Geoffrey E Hinton. Rectified linear units improve restricted boltzmann machines. In *Proceedings of the 27th international conference on machine learning (ICML-10)*, pages 807–814, 2010.
- [31] Morgan Quigley, Ken Conley, Brian Gerkey, Josh Faust, Tully Foote, Jeremy Leibs, Rob Wheeler, and Andrew Ng. Ros: an open-source robot operating system. volume 3, 01 2009.
- [32] Nathan D Ratliff, J Andrew Bagnell, and Martin A Zinkevich. Maximum margin planning. In *Proceedings of the 23rd international conference on Machine learning*, pages 729–736, 2006.
- [33] Eric M. Refour, Bijo Sebastian, Raghuraj J. Chauhan, and Pinhas Ben-Tzvi. A general purpose robotic hand exoskeleton with series elastic actuation. *Journal of Mechanisms and Robotics*, 11(6):060902, December 2019. Epub 2019 Sep 10.

- [34] Eric Rohmer, Surya P. N. Singh, and Marc Freese. V-rep: A versatile and scalable robot simulation framework. In *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 1321–1326, 2013.
- [35] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning representations by back-propagating errors. *nature*, 323(6088):533–536, 1986.
- [36] Ken Shoemake. Animating rotation with quaternion curves. In *Proceedings of the 12th annual conference on Computer graphics and interactive techniques*, pages 245–254, 1985.
- [37] Kenta Takaya, Toshinori Asai, Valeri Kroumov, and Florentin Smarandache. Simulation environment for mobile robots testing using ros and gazebo. pages 96–101, 10 2016.
- [38] E. Todorov, T. Erez, and Y. Tassa. Mujoco: A physics engine for model-based control. In *IEEE International Conference on Intelligent Robots and Systems*, pages 5026–5033. IEEE, October 2012.
- [39] A. M. TURING. I.—computing machinery and intelligence. *Mind*, LIX(236):433–460, 10 1950.
- [40] Michael Wiertlewski, José Lozada, and Vincent Hayward. The spatial spectrum of tangential skin displacement can encode tactual texture. *Robotics, IEEE Transactions on*, 27:461 – 472, 07 2011.
- [41] Jaemin Yoon, Bukun Son, and Dongjun Lee. Comparative study of physics engines for robot simulation with mechanical interaction. *Applied Sciences*, 13(2), 2023.
- [42] Brian D Ziebart, Andrew L Maas, J Andrew Bagnell, Anind K Dey, et al. Maximum entropy inverse reinforcement learning. In *Aaai*, volume 8, pages 1433–1438. Chicago, IL, USA, 2008.